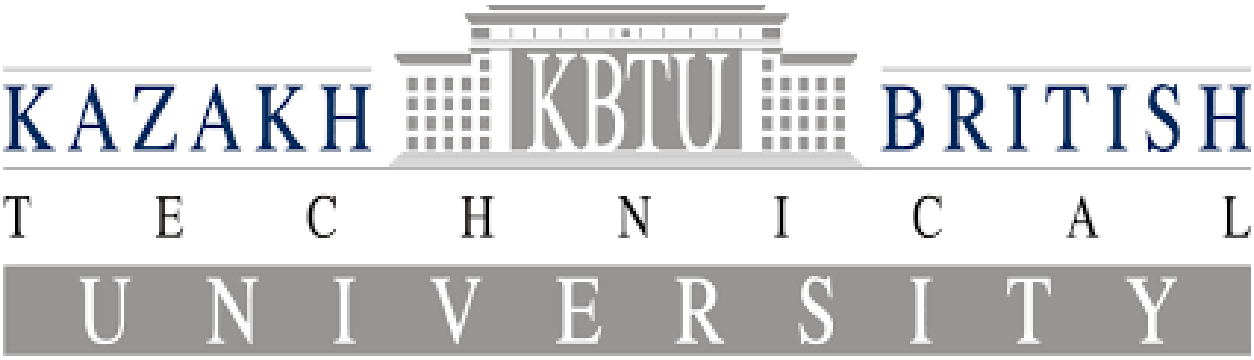

School of Information Technology and Engineering

Security of Operating System

SIS 1

Project Introduction and Machines Provisioning

Checked by: Valyayev Denis

Done by: Faramarz Pedram

Almaty, 2026

Target

The target of this Individual Study is to define and describe the selected individual project, design a high-level architecture of the system, and prepare the required machines for further implementation. This study represents the initial phase of the project and focuses on planning, architectural design, and infrastructure provisioning only.

1. Project Selection and Description

Project Name:

Secure SWIFT Logical Terminal Infrastructure

Project Legend

A new bank is preparing its internal infrastructure before connecting it to the international SWIFT system.

Before any external connectivity is allowed, the bank must deploy a secure internal SWIFT terminal environment that will store and process highly confidential financial files. In the scope of this project, the SWIFT terminal is implemented as a logical file-transfer and file-storage system operating on the local Linux file system. The files stored in the system contain sensitive financial information and must be protected from unauthorized access, theft, and modification.

The project does not implement real SWIFT protocols or software. Instead, it focuses on designing a **secure operating system environment** that is suitable for such a system, following best practices of operating system security.

Main Project Target

The main target of the project is to design and prepare a Linux-based infrastructure that can support a secure SWIFT logical terminal system, with a strong emphasis on operating system security, system isolation, auditability, and reliability.

Project Tasks

The main tasks of the project are:

- To analyze the security requirements of a SWIFT terminal environment
- To design a secure system architecture with clearly separated responsibilities
- To identify all required machines and planned software components
- To choose an appropriate machine provisioning method
- To select and justify a Linux distribution
- To prepare the required machines with operating systems installed

2. Project Architecture

Architecture Overview

The designed system architecture consists of two Linux-based servers, each with a clearly defined role.

Separating system responsibilities across multiple machines is a common security practice in financial and government infrastructures and improves system security, monitoring, and recovery capabilities.

At this stage, the architecture is defined at a conceptual level. Actual configuration of services and security mechanisms will be performed in subsequent project stages.

2.1 Machines list (planned infrastructure)

The project consists of two Fedora Linux virtual machines:

1. swift-terminal — SWIFT Terminal Server (main system)
2. swift-support — Supporting Services Server (security and reliability services)

This separation is chosen to support security requirements: logs and backups must be stored separately from the main SWIFT terminal system.

2.2 Software units per machine (what will be installed)

Machine 1: swift-terminal (SWIFT Terminal Server, Fedora)

Goal: Provide secure file-transfer and file-storage environment (logical SWIFT terminal) and a controlled interface for operators.

Software units planned for installation:

1. Web/Reverse Proxy + Frontend serving
    - Nginx (serves static frontend files and forwards API requests to backend)
2. Frontend (UI)
    - Static web UI files (HTML/CSS/JS) hosted by Nginx
3. Backend API (business logic)
    - Python 3 + FastAPI (or Flask)
    - Uvicorn/Gunicorn (application server)
    - Runs as a dedicated Linux service user (planned)
4. Database server (metadata)
    - PostgreSQL
    - Stores metadata only (filename, hash, timestamps, status, user actions)
5. Local SWIFT terminal storage (file system unit)
    - Secure LT directories: LT1–LT5 under /swift-lt/
6. System logging agent
    - journald + rsyslog client (to forward logs to support server)
7. System security tooling (planned)

- firewalld
- SELinux (Fedora default)
- OpenSSH (restricted access)

note: in this phase, only OS is installed. The above software units are defined as planned components and will be installed/configured later.

Machine 2: swift-support (Supporting Services Server, Fedora)

Goal: Provide separate storage and security services: centralized logging, backups, and monitoring/alerting.

Software units planned for installation:

1. Centralized logging receiver
    - rsyslog server (receives logs from swift-terminal)
2. Backup repository
    - rsync + tar for backup storage
    - Encrypted backup archives (planned)
3. Monitoring and alerting
    - Basic alerting service for login events (e.g., email/telegram/local notification script — final tool to be selected later)
4. Administrative access control
    - OpenSSH for admin access (restricted)
    - Acts as a secure admin entry point (jump host concept)
5. System security tooling (planned)
    - firewalld
    - SELinux

Note: in this phase, only OS is installed. The above software units are planned.

Architecture Diagram (Conceptual Description)

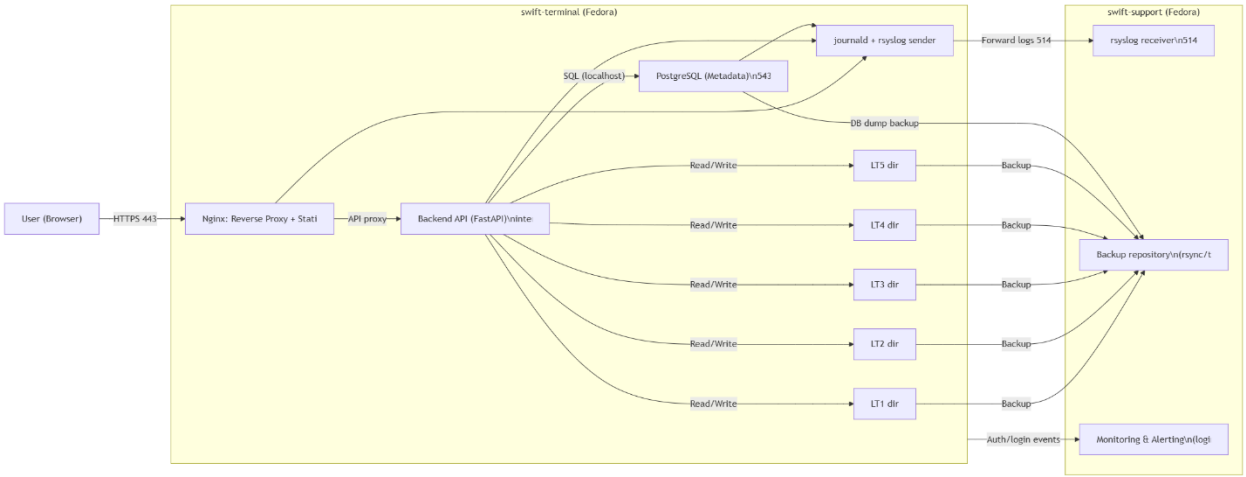

Diagram explanation (in words):

- Users access the system only via HTTPS to Nginx.
- Nginx serves the frontend and forwards API calls to the backend.
- Backend writes metadata to PostgreSQL and stores confidential files in LT1–LT5 directories on the local file system.
- Logs from services and OS are sent to the support server's log receiver.
- Backups of LT directories and database dumps are stored on the support server.
- Monitoring/alerting is handled on the support server.

3. Machines Provisioning

Provisioning Method

The machines for this project were created using Virtual Machines (VMs).

Justification:

- Virtual machines provide isolation from the host operating system
- They allow safe experimentation with security configurations
- Snapshots enable easy recovery from configuration errors
- This approach is recommended for operating system security studies

Linux Distribution Choice

Chosen Distribution: Fedora Linux

Justification:

- Fedora is a modern and secure Linux distribution
- It provides strong SELinux support by default
- It is closely related to RHEL, which is preferred for enterprise and government infrastructures
- Fedora is suitable for learning and implementing advanced operating system security mechanisms

Prepared Machines:

Machine Name	Role	Status
swift-terminal	SWIFT Terminal Server	OS installed
swift-support	Supporting Services Server	OS installed

At this stage, both machines are prepared with operating systems installed and are ready for further configuration.

4. Task Fulfillment (Step-by-Step)

1. The SWIFT logical terminal project was selected and analyzed

2. Project legend, objectives, and tasks were defined
3. A high-level system architecture was designed
4. Required machines and software units were identified
5. Virtual machines were chosen as the provisioning method
6. Fedora Linux was selected and justified
7. Two virtual machines were created and operating systems were installed

4. **Architecture Strengths: Security Controls by Layer (extra)**

1. Physical / Virtualization Layer (VMs, host isolation)

Security measures

- Virtual Machines (VMs) for isolation
- No direct access to host OS
- Snapshots for rollback
- Separate VMs for SWIFT and support services

Why it matters

- Limits damage if one system is compromised
- Prevents experiments from affecting host machine
- Supports recovery after misconfiguration or attack

Strength: Strong isolation boundary

2. Operating System Layer (Fedora Linux)

Security measures

- Root login disabled
- Sudo-based privilege escalation
- Dedicated Linux users per service
- Strong password policies
- Account lockout rules
- Last-login information display

- SELinux enforcing mode
- Secure file permissions (chmod/chown)
- Full-disk encryption (LUKS)

Why it matters

- Prevents privilege escalation
- Enforces least privilege
- Protects data even if disk is stolen
- Controls access even if application fails

Strength: Core security enforced at OS level, not application level

3. Network Layer (Traffic control and isolation)

Security measures

- firewalld rules
- Only required ports open (e.g. 443, 22)
- Database accessible only locally
- Log forwarding allowed only to support server
- No direct access to backup storage
- Admin access restricted (jump host model)

Why it matters

- Reduces attack surface
- Prevents lateral movement
- Ensures only expected communication paths exist

Strength: Explicitly controlled communication paths

4. Application Layer – Frontend (User interface)

Security measures

- HTTPS only
- No direct file-system access
- Static content only
- Runs as non-root user
- No sensitive logic in frontend

Why it matters

- Frontend compromise does not expose files
- Limits impact of XSS or UI-level attacks

Strength: Frontend is isolated and untrusted by design

5. Application Layer – Backend (Control & enforcement point)

Security measures

- Runs as dedicated Linux service user
- Validates user actions
- Enforces role-based access
- Controls all file access
- Logs every sensitive action
- No root privileges
- Limited directory permissions

Why it matters

- Centralized enforcement of rules
- Prevents unauthorized file operations
- Provides traceability

Strength: Backend acts as a controlled security gate

6. Data Layer – Database (Metadata only)

Security measures

- Stores metadata, not financial files
- Local-only access
- Dedicated DB user
- No external exposure
- Regular backups

Why it matters

- Data breach impact is minimized
- Attackers cannot steal real financial files from DB

Strength: Sensitive data never enters the database

7. File System Layer – SWIFT Logical Terminal (Most sensitive data)

Security measures

- Stored on encrypted disk
- 5 isolated LT directories
- Strict ownership and permissions
- Access only via backend service
- No direct user access
- Regular backups

Why it matters

- Protects confidentiality and integrity
- Prevents direct access even if user is compromised

Strength: File system is the final protected vault

8. Logging & Auditing Layer (Accountability & detection)

Security measures

- journald + rsyslog
- Centralized log forwarding
- External log storage
- Login event tracking
- Immutable logs on support server

Why it matters

- Detects attacks
- Supports forensic analysis
- Meets regulatory requirements

Strength: Actions are traceable and tamper-resistant

9. Backup & Recovery Layer (Availability & resilience)

Security measures

- Scheduled backups
- Encrypted backup archives
- Stored on separate server
- Tested restore procedures

Why it matters

- Protects against ransomware
- Enables recovery after failure or compromise

Strength: Availability and resilience are guaranteed

10. Administrative Layer (Human access control)

Security measures

- SSH key-based access
- Restricted admin users
- Jump-host model
- Full audit of admin actions

Why it matters

- Admin accounts are common attack targets
- Controls insider threats

Strength: Admin access is controlled and auditable

6. Conclusions

In this Individual Study, a secure SWIFT logical terminal project was selected and clearly defined.

A high-level system architecture was designed, identifying all machines and planned software components.

The machine provisioning method and Linux distribution were chosen and justified based on security and educational goals.

Two virtual machines were successfully prepared with operating systems installed, forming a solid foundation for further project implementation.

Furthermore, the implementation of defense-in-depth will add an extra layer of protection by combining operating system security, network isolation, application-level controls, and external logging, significantly reducing the risk of a single point of failure.